# Progress Report 2

**Student Name:** Reubin Chatta

**Student ID:** 300394193

**Project:** Smart Door Anomaly Detection & Cybersecurity for Elderly Care

**Course**: CSIS 4495 – Applied Research Project, Section 3

**Reporting Period:** Feb 24 – March 2, 2026

**Report Date:** March 2, 2026

---

**Work Date/Hours Logs**

| Date | Hours | Description |
|---|---|---|
| Feb 27, 2026 | 3 | Worked on critical linking of isolation forest API \| Researched Designed Flask startup sequence for model loading |
| Mar 1, 2026 | 3 | Implemented model loading at Flask initialization with error handling and logging \| Integrated anomaly scoring into /recognize endpoint; added anomaly score and is anomaly to response |
| Mar 2, 2026 | 2.5 | Anomaly logging, threat alert generation, dashboard integration, and testing \| Team meeting |
| **Total** | 8.5 | |

---

**Summary Description of Work Done**

This week, I completed the critical integration of the trained Isolation Forest anomaly detection model into the live facial recognition API. The model had been developed and trained but was never actually called by the system.

**The Problem:** The /recognize endpoint executed facial recognition and logged access decisions, but **zero anomaly detection was running in production**. Recognition events were processed without checking if they were anomalous.

**The Solution:**

1. **Model Loading at Startup** (api/**init**.py): Modified Flask initialization to load the trained Isolation Forest model and StandardScaler at server startup with error handling.
2. **Real-Time Anomaly Scoring** (api/routes.py): Every successful recognition event now flows through the model, computing an anomaly score (0–1 range).
3. **Response Enhancement:** Updated API response to include:
    - anomaly_score: Numerical score from model
    - is_anomaly: Boolean flag for anomalous events
4. **Anomaly Logging & Alerts:**
    - Anomalous events logged to database anomalies table
    - MEDIUM severity threat alerts automatically generated
    - Alert includes person name, confidence, anomaly score, timestamp
5. **Dashboard Integration:** Frontend threat detection now displays anomaly-based alerts alongside traditional violations.

**Issues Encountered:**

- Model trained but completely disconnected from production pipeline
- No API response fields for anomaly information
- No alerting mechanism for suspicious patterns

---

**Repo Check In of Implementation Completed**

Files checked in this period:

- **api/init.py** — Flask app initialization with model loading, error handling, logging
- **api/routes.py** — Modified /recognize endpoint to call model.predict() and handle anomaly scores
- **Database schema** — Added anomalies table with person_id, timestamp, anomaly_score, confidence_score, alert_id
- **Dashboard integration** — Threat alert system updated to display anomaly-based alerts
- **API_DOCS.md** — Updated with new response fields and anomaly detection behavior

System now provides end-to-end observable anomaly detection from model inference through dashboard visualization.

---

**AI Use Section**

| AI Tool | Version / Account Type | Specific Use | Value Addition |
|---|---|---|---|
| GitHub Copilot | Free Tier | Flask app initialization patterns and model loading | Provided basic template; enhanced with custom error handling, logging, and model versioning for our use case |
| ChatGPT 5.3 | Pro Tier | Anomalies table schema and REST API response contract | Suggested generic structures; tailored schema for security audit requirements and integrated with threat alert system |

---

**Appendix: Prompt History**

**Prompt 1:** "How do I load a scikit-learn model at Flask startup and make it available to all endpoints?"

- **Used:** Model load in app factory pattern
- **Modified:** Added custom exception handling, logger for debugging, and model staleness warnings

**Prompt 2:** "What fields should a facial recognition API response include for anomaly detection?"

- **Used:** Basic structure (person_id, confidence, timestamp)
- **Modified:** Added anomaly fields (anomaly_score, is_anomaly), linked to threat alert infrastructure, ensured backward compatibility

**Prompt 3:** "Design a database table for logging security anomalies with good performance"

- **Used:** Generic table structure with timestamp and score
- **Modified:** Added foreign keys to persons/alerts tables, compound indexing on (person_id, timestamp), added 6-month retention policy

**Prompt 4:** "Write code to auto-generate security alerts when anomaly_score exceeds threshold"

- **Used:** If-statement threshold checking pattern
- **Modified:** Made threshold configurable via environment variable, added deduplication (suppress duplicates within 5 min), integrated with severity levels